

PERSONAL FINANCE TRACKER

Project Report

Technology: Java 17+

Application Type: Command-Line Application

Data Storage: Local CSV Files

1. Introduction

Personal financial management is an important part of everyday life. Users need to keep track of their income, expenses, budgets, and spending patterns to understand their financial position and make better decisions.

The **Personal Finance Tracker** is a Java-based command-line application developed to provide a simple and lightweight solution for managing personal finances. The application allows users to record income and expenses, edit or delete transactions, set category-wise monthly budgets, and generate financial reports.

The application uses **Java 17+** and stores data in local CSV files instead of a database. This makes the system portable and easy to run because it does not require a database server, external libraries, Maven, or Gradle.

The project follows a layered architecture consisting of model, service, storage, and utility components. This improves maintainability and keeps different responsibilities separated.

2. Problem Statement

Managing personal finances manually can become difficult as the number of transactions increases. Users may find it difficult to remember their expenses, calculate monthly spending, monitor category budgets, and compare spending between different months.

Traditional finance applications may also require databases, accounts, internet connectivity, or complicated setup.

The objective of this project is to develop a simple command-line **Personal Finance Tracker** that allows users to:

- Record income and expenses.
 - View all financial transactions.
 - Edit and delete existing transactions.
 - Set monthly budgets for different categories.
 - Check spending against a budget.
 - Generate monthly financial summaries.
 - Analyze category-wise spending.
 - Compare spending between months.
 - Store data locally using CSV files.
-

3. Functional Requirements

The project guidelines require at least three major functional modules, clear input/output, and a logical workflow.

The Personal Finance Tracker provides the following functional requirements.

3.1 Transaction Management

The application allows users to:

- Add a new transaction.
- View all transactions.
- Edit an existing transaction.
- Delete a transaction.
- Retrieve transactions by month.
- Retrieve transactions by category.

Each transaction contains:

- Transaction ID
- Date
- Transaction type
- Category
- Amount
- Description

Transaction types are:

- INCOME
- EXPENSE

3.2 Budget Management

Users can set a monthly spending limit for a category.

The system compares actual spending with the defined budget and returns one of three statuses:

- **OK** – spending is below 80% of the budget.
- **WARNING** – spending is at least 80% of the budget.
- **OVER BUDGET** – spending exceeds the budget.

3.3 Financial Reporting

The application provides:

- Total income.
- Total expenses.
- Net balance.
- Category-wise spending percentages.
- Month-over-month spending trend.

3.4 Input Validation

The application validates:

- Dates in `yyyy-MM-dd` format.
- Months in `yyyy-MM` format.
- Positive transaction amounts.
- Non-empty strings.

Invalid input does not terminate the application. Instead, the user is prompted to enter the value again.

3.5 Data Persistence

Transaction and budget information is stored in CSV files.

The application immediately persists changes after write operations.

3.6 Logging

The application uses Java's built-in `java.util.logging` package to record important events such as:

- Adding transactions.
- Editing transactions.
- Deleting transactions.
- Errors.

The log is stored in `app.log`.

4. Non-functional Requirements

The VITyarthi guidelines require at least four non-functional requirements.

4.1 Usability

The application provides a simple numbered command-line menu. Users can select operations using numbers and receive clear prompts.

4.2 Reliability

The application includes exception handling and a top-level safety net in the `Main` class to prevent unexpected termination.

4.3 Maintainability

The project uses separate packages for models, services, storage, and utilities. This makes the code easier to understand, modify, and extend.

4.4 Portability

The project uses only standard Java 17+ features and has no external dependencies.

4.5 Error Handling

Invalid user input is detected through the centralized `InputValidator` class. The application re-prompts the user instead of crashing.

4.6 Resource Efficiency

The application uses local CSV files instead of requiring a database server or other infrastructure.

4.7 Logging

Important application events and errors are recorded in `app.log` using the Java standard logging framework.

5. System Architecture

Architecture Components

Main:

Responsible for the command-line interface, menu loop, user input, and interaction with services.

Model Layer:

Contains classes representing application data.

Service Layer:

Contains the application's business logic.

Storage Layer:

Provides reusable CSV reading, writing, and appending functionality.

Utility Layer:

Provides centralized validation and logging functionality.

The layered approach supports modular and maintainable implementation, which is one of the technical expectations in the supplied guidelines.

Main Use Cases

The user can:

1. Add a transaction.
2. View transactions.
3. Edit a transaction.
4. Delete a transaction.
5. Set a budget.

6. View budget status.
 7. View monthly summary.
 8. View category breakdown.
 9. View spending trend.
-
-
-

6. Storage Design

The application uses CSV files rather than a relational database.

The supplied guidelines mention an ER diagram when database/storage design is applicable. Since this project uses CSV storage, the file schema is used to represent the storage design.

transactions.csv

id,date,type,category,amount,description

Example:

1,2026-09-01,INCOME,Salary,50000,Monthly salary
2,2026-09-02,EXPENSE,Food,500,Lunch

budgets.csv

category,monthlyLimit

Example:

Food,5000
Transport,3000
Entertainment,2000

7. Design Decisions and Rationale

Java 17+

Java 17+ was selected because it provides a stable modern Java environment while keeping the application portable.

CSV Instead of Database

CSV was selected because the project requires zero setup. Users do not need to install or configure a database.

CSV files are also:

- Easy to read.
- Easy to back up.
- Lightweight.
- Portable.

Layered Architecture

The project separates the application into model, service, storage, and utility layers. This prevents all logic from being placed inside the main CLI class.

Manual Testing

`AppTest` uses manual assertions rather than JUnit. This satisfies the dependency-free requirement while still providing a simple testing mechanism.

Standard Java Logging

`java.util.logging` was selected instead of an external logging library to maintain the zero-dependency requirement.

Immediate Persistence

Changes are written to CSV immediately after modification so that important data is not kept only in memory.

18. Implementation Details

8.1 Transaction

The `Transaction` class represents a financial transaction.

It contains:

- `id`
- `date`
- `type`
- `category`
- `amount`
- `description`

The class provides `toCsvRow()` and `fromCsvRow()` methods for CSV serialization.

8.2 TransactionType

`TransactionType` is an enum containing:

INCOME
EXPENSE

This prevents invalid transaction types from being introduced.

8.3 Budget

The `Budget` class contains:

- Category.
- Monthly spending limit.

It also provides CSV conversion methods.

8.4 TransactionService

The `TransactionService` manages transaction data and provides:

- Add.
- Edit.
- Delete.
- Get all.
- Get by month.
- Get by category.

Transaction IDs are automatically incremented.

8.5 BudgetService

The **BudgetService** manages category budgets and determines the budget status.

The status rules are:

Spending < 80% → OK
Spending >= 80% → WARNING
Spending > 100% → OVER BUDGET

8.6 ReportService

The reporting service calculates:

- Total income.
- Total expenses.
- Net balance.
- Category-wise spending percentages.
- Month-over-month spending trend.

8.7 CsvStorage

CsvStorage is a reusable component responsible for reading, writing, and appending CSV data.

Both transaction and budget services use this component.

8.8 InputValidator

The validator ensures that user input follows the required format.

Examples include:

Date: yyyy-MM-dd

Month: yyyy-MM

Amount: Positive number

String: Non-empty

8.9 AppLogger

AppLogger configures Java's standard logging framework and writes application events to:

app.log

9. Screenshots / Results

```
=====
                        FINANCE TRACKER AUTOMATED TESTS
=====

-- Testing Transaction Model & CSV Roundtrip --
[PASS] Transaction ID preserves in CSV
[PASS] Transaction Date preserves in CSV
[PASS] Transaction Type preserves in CSV
[PASS] Transaction Category preserves in CSV
[PASS] Transaction Amount preserves in CSV
[PASS] Transaction Description with comma preserves in CSV

-- Testing Budget Model & CSV Roundtrip --
[PASS] Budget Category preserves in CSV
[PASS] Budget Limit preserves in CSV

-- Testing InputValidator --
[PASS] Valid date format yyyy-MM-dd
[PASS] Invalid date format
[PASS] Invalid leap year date
[PASS] Valid month format yyyy-MM
[PASS] Invalid month format
[PASS] Valid positive amount
[PASS] Zero amount is rejected
[PASS] Negative amount is rejected
[PASS] Non-empty string is valid
[PASS] Whitespace-only string is rejected

-- Testing TransactionService CRUD & Auto-Increment --
[PASS] Auto-increment ID for first item
[PASS] Auto-increment ID for second item
[PASS] Auto-increment ID for third item
[PASS] Total transaction count is 4
[PASS] Edit transaction returns true
[PASS] Edited transaction found
[PASS] Updated amount is reflected
[PASS] August transactions count
[PASS] September transactions count
[PASS] Salary transactions count across months
[PASS] Delete transaction 1 succeeded
[PASS] Transaction count after delete
[PASS] Reloaded service has matching transaction count
[PASS] Deleted transaction is not in reloaded service
[PASS] Remaining transaction 2 exists in reloaded service
[PASS] New transaction receives id 5 after re-open

-- Testing BudgetService Thresholds (OK, WARNING, OVER BUDGET) --
[PASS] Report contains 3 budget items
[PASS] Rent status is OK
[PASS] Food status (85.7%) is WARNING
[PASS] Utilities status (150%) is OVER_BUDGET

-- Testing ReportService Summaries & Breakdown --
[PASS] Sept Total Income
[PASS] Sept Total Expense
[PASS] Sept Net Balance
[PASS] Breakdown contains 2 expense categories
[PASS] Top expense category is Food
[PASS] Food percentage is 66.6667%
[PASS] Second expense category is Utilities
[PASS] Utilities percentage is 33.3333%

-- Testing Month-over-Month Trend Calculation --
[PASS] Aug (previous) expense
[PASS] Sept (current) expense
[PASS] Absolute change
[PASS] Percentage change is non-null
[PASS] Percentage change is -64%
=====
Test Execution Summary: Total: 51 | Passed: 51 | Failed: 0
=====
ALL TESTS PASSED SUCCESSFULLY [OK]
```

```
C:\Users\HUZAIFA PANWAR\OneDrive\ドキュメント\Desktop\vityarthi projec\finance tracker org>java -cp bin com.financetracker.Main
```

```
=====
```

```
PERSONAL FINANCE TRACKER (CLI)
```

```
Core Java SE 17+ | Zero Dependencies | Local CSV
```

```
=====
```

```
----- MAIN MENU -----
```

1. Add Transaction
2. View All Transactions
3. Edit a Transaction
4. Delete a Transaction
5. Set a Category Budget
6. View Budget Status for a Month
7. Monthly Income / Expense Summary
8. Category-wise Spend Breakdown
9. Month-over-Month Spending Trend
0. Exit

```
----- MAIN MENU -----
```

1. Add Transaction
2. View All Transactions
3. Edit a Transaction
4. Delete a Transaction
5. Set a Category Budget
6. View Budget Status for a Month
7. Monthly Income / Expense Summary
8. Category-wise Spend Breakdown
9. Month-over-Month Spending Trend
0. Exit

```
-----
```

```
Enter your choice [0-9]: 1
```

```
--- Add New Transaction ---
```

```
Date (yyyy-MM-dd) [2026-09-07]: 2026-04-05
```

```
Type (INCOME or EXPENSE): income
```

```
Category (e.g. Salary, Food, Utilities): salary
```

```
Amount ($): 1000
```

```
Description (optional):
```

```
[OK] Success: Added INCOME transaction #2 for $1000.00 under 'salary'.
```

```
----- MAIN MENU -----
1. Add Transaction
2. View All Transactions
3. Edit a Transaction
4. Delete a Transaction
5. Set a Category Budget
6. View Budget Status for a Month
7. Monthly Income / Expense Summary
8. Category-wise Spend Breakdown
9. Month-over-Month Spending Trend
0. Exit
-----
Enter your choice [0-9]: 2

--- All Recorded Transactions ---
+-----+-----+-----+-----+-----+-----+
+-----+
| ID | Date       | Type   | Category   | Amount ($) | Description |
+-----+-----+-----+-----+-----+-----+
+-----+
| 2  | 2026-04-05 | INCOME | salary     | $ 1000.00 |             |
+-----+-----+-----+-----+-----+-----+
| 1  | 2026-09-03 | INCOME | salary     | $ 1000.00 |             |
+-----+-----+-----+-----+-----+-----+
+-----+
Total records: 2
```

```
-----
Enter your choice [0-9]: 3

--- Edit Transaction ---
Enter Transaction ID to edit: 1
Editing current record: Transaction #1 | 2026-09-03 | INCOME | salary
| $1000.00 |
(Press Enter to keep the current value in brackets)
Date [2026-09-03]: 500
[!] Error: Please enter a valid date in yyyy-MM-dd format.
Date [2026-09-03]: 2026-04-05
Type [INCOME]: 500
[!] Error: Type must be either 'INCOME' or 'EXPENSE'.
Type [INCOME]: income
Category [salary]: 500
Amount [1000.00]:
Description:
[OK] Success: Transaction #1 has been updated.
```

----- MAIN MENU -----

1. Add Transaction
2. View All Transactions
3. Edit a Transaction
4. Delete a Transaction
5. Set a Category Budget
6. View Budget Status for a Month
7. Monthly Income / Expense Summary
8. Category-wise Spend Breakdown
9. Month-over-Month Spending Trend
0. Exit

Enter your choice [0-9]: 4

--- Delete Transaction ---

Enter Transaction ID to delete: 1

Target transaction: Transaction #1 | 2026-04-05 | INCOME | 500
| \$1000.00 |

Are you sure you want to delete this transaction? (y/N): y

[OK] Success: Transaction #1 deleted.

1. Add Transaction
2. View All Transactions
3. Edit a Transaction
4. Delete a Transaction
5. Set a Category Budget
6. View Budget Status for a Month
7. Monthly Income / Expense Summary
8. Category-wise Spend Breakdown
9. Month-over-Month Spending Trend
0. Exit

Enter your choice [0-9]: 5

--- Set Category Monthly Budget ---

Category name (e.g. Groceries, Rent, Dining): household

Monthly budget limit (\$): 300

[OK] Success: Monthly budget for 'household' set to \$300.00.

Enter your choice [0-9]: 6

--- View Monthly Budget Status ---

Enter month (yyyy-MM) [2026-09]: 2026-04-05

Invalid format. Re-enter month (yyyy-MM): 2026-04

Budget Performance Report for 2026-04:

Category	Monthly Limit	Actual Spend	Remaining	% Used	Status
household	\$ 300.00	\$ 0.00	\$ 300.00	0.0%	[OK]
household	\$ 200.00	\$ 0.00	\$ 200.00	0.0%	[OK]

Enter your choice [0-9]: 7

--- Monthly Income / Expense Summary ---

Enter month (yyyy-MM) [2026-09]: 2026-10

Financial Summary for 2026-10 (0 transactions):

Total Income:	\$	0.00
Total Expenses:	\$	0.00
Net Balance:	\$	0.00 (Surplus)

Enter your choice [0-9]: 8

--- Category-wise Spend Breakdown ---

Enter month (yyyy-MM) [2026-09]: 2026-04

No expense transactions recorded for 2026-04.

Enter your choice [0-9]: 9

--- Month-over-Month Spending Trend ---

Enter target month (yyyy-MM) [2026-09]: 2026-04

Spending Trend Comparison: 2026-04 vs. 2026-03

Previous Month (2026-03) Spending:	\$	0.00
Current Month (2026-04) Spending:	\$	0.00
Absolute Difference:	\$	0.00
Percentage Change:		0.00%

Insight: No spending recorded in either 2026-03 or 2026-04.

10. Testing Approach

The project uses `AppTest`, a dependency-free test class containing manual assertions. The VITyarthi guidelines specifically require testing wherever applicable.

The following areas should be tested:

Test	Expected Result
Transaction CSV serialization	Data can be converted to and from CSV
Budget CSV serialization	Budget information is preserved
Date validation	Invalid dates are rejected
Month validation	Invalid months are rejected
Amount validation	Zero/negative amounts are rejected
String validation	Empty strings are rejected
Add transaction	Transaction is stored successfully
Edit transaction	Existing transaction is updated
Delete transaction	Transaction is removed
Budget status	Correct status is returned
Monthly summary	Income/expense totals are correct
Category breakdown	Percentages are calculated correctly
Spending trend	Month-over-month percentage is calculated
CLI flow	Complete workflow runs without crashing

The application should also be tested using the required end-to-end sequence:

1. Start the application.
2. Add transaction 1.
3. Add transaction 2.
4. Set a category budget.
5. View monthly summary.
6. View category breakdown.
7. View spending trend.

8. Exit.

11. Challenges Faced

Several challenges were encountered during the development of the project.

CSV Data Management

Managing structured data using CSV files required careful handling of reading, writing, appending, and converting Java objects into CSV rows.

Automatic ID Generation

The application needs to maintain unique transaction IDs while supporting additions and deletions.

Input Validation

Command-line applications receive input directly from users, so invalid values need to be handled without terminating the program.

Budget Calculations

The application needs to correctly determine whether spending is within the budget, approaching the limit, or exceeding it.

Reporting

Calculating category percentages and month-over-month spending requires processing transaction data accurately.

Separation of Responsibilities

Another challenge was maintaining clean separation between CLI interaction, business logic, storage, validation, and logging.

12. Learnings and Key Takeaways

This project provided practical experience in Java application development and software architecture.

Major learnings include:

- Applying object-oriented programming to a real-world problem.
 - Designing a layered software architecture.
 - Working with Java collections and file handling.
 - Implementing CSV-based persistence.
 - Creating reusable service and storage components.
 - Implementing CRUD operations.
 - Designing validation and error-handling mechanisms.
 - Using Java's standard logging framework.
 - Writing dependency-free manual tests.
 - Understanding separation of concerns.
 - Organizing a project using packages.
 - Using Git and GitHub for version control and project sharing.
-

13. Future Enhancements

The current application can be extended with several features.

Graphical User Interface

A GUI could make the application more accessible to users who prefer visual interfaces.

Web Application

The service layer could be reused to build a web-based finance management system.

Database Support

A future version could use MySQL, PostgreSQL, or another database for handling larger datasets.

Charts and Visualization

Charts could be added to visualize:

- Monthly expenses.
- Category spending.

- Income versus expenses.
- Budget utilization.

Recurring Transactions

Users could create recurring transactions such as monthly salary, rent, subscriptions, and bills.

Data Export

The application could support exporting reports to PDF or Excel formats.

Authentication

A multi-user version could provide user accounts and authentication.

Advanced Budgeting

Future versions could support:

- Yearly budgets.
- Multiple budgets per category.
- Budget history.
- Budget recommendations.

14. Conclusion

The **Personal Finance Tracker** is a lightweight Java 17+ command-line application designed to simplify personal financial management.

The application provides transaction management, category budgeting, financial reporting, CSV persistence, input validation, error handling, and logging. Its layered architecture separates the user interface from business logic and storage, making the system easier to maintain and extend.

The project demonstrates practical application of Java programming, object-oriented design, file handling, validation, testing, and software architecture concepts.

By avoiding external dependencies and databases, the application remains simple to install and portable across systems with a compatible JDK.
